Лабораторийн ажил №6

Синхрончлолын хэрэгслүүд — Бүлэг 6, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Дасгал 6.1

Асуулт. 6.4-р хэсэгт тасалдлыг (interrupt) байнга хаах нь системийн цагт нөлөөлж болно гэж дурдсан. Энэ нь яагаад тохиолддогийг болон ийм нөлөөг хэрхэн багасгаж болохыг тайлбарла.

Хариулт.

Шалтгаан. Системийн цагийг (system clock) ихэвчлэн timer тасалдал шинэчилдэг. Тодорхой давтамжтайгаар (жнь 1 мс тутам) тасалдал ирэхэд кернел цагийн тоолуурыг нэмэгдүүлдэг. Эгзэгтэй мужид орохын тулд тасалдлыг хаасан үед timer тасалдал хүлээгдэж, хоцорно. Хэрэв хаалт удаан үргэлжилж, хэд хэдэн timer тасалдал ирвэл техник хангамж ихэвчлэн зөвхөн нэгийг нь хадгалдаг тул бусад нь алдагдана. Ингэснээр системийн цаг бодит цагаас хоцорно. Мөн CPU хуваарилалтын квант, процессын CPU хэрэглээний бүртгэл (accounting) буруу болно.

Багасгах арга:

- Тасалдлыг зөвхөн маш богино хугацаагаар (хэдхэн заавар) хааж, аль болох хурдан нээх. Эгзэгтэй мужийг жижиг байлгах.
- Тасалдал хаахын оронд атомар заавар (`test_and_set`, `compare_and_swap`), spinlock, мутекс зэрэг бусад синхрончлолын хэрэгсэл хэрэглэх. Олон процессортой системд тасалдал хаах нь бүх CPU-д үйлчлэхгүй тул үр ашиггүй.
- Цагийг зөвхөн тасалдал тоолохоор биш, тогтвортой техник хангамжийн цаг (RTC, TSC, HPET)-аас үе үе уншиж засах. Ингэснээр алдагдсан tick нөлөөлөхгүй.

Дасгал 6.2

Асуулт. Завгүй хүлээлт (busy waiting) гэж юуг хэлэх вэ? Үйлдлийн системд өөр ямар төрлийн хүлээлт байдаг вэ? Завгүй хүлээлтээс бүрэн зайлсхийж болох уу? Хариултаа тайлбарла.

Хариулт.

Завгүй хүлээлт гэдэг нь процесс эсвэл урсгал нөхцөл биелэхийг давталтаар тасралтгүй шалгаж хүлээхийг хэлнэ. Жишээ нь `while (!available) ;` гэх мэт. Энэ хугацаанд CPU ашигтай ажил хийхгүй, зөвхөн "эргэлдэнэ" (spin). Spinlock нь завгүй хүлээлттэй түгжээ юм.

Бусад төрлийн хүлээлт нь блоклогдох (унтах) хүлээлт юм. Процесс CPU-г чөлөөлж хүлээлтийн төлөвт шилжиж, хүлээлтийн дараалалд орно. Жишээ нь семафорын хүлээлтийн жагсаалт, I/O дуусахыг хүлээх, condition variable дээр `wait()`, `sleep()`. Нөхцөл биелэхэд өөр процесс эсвэл кернел түүнийг сэрээж (`signal`/`wakeup`) бэлэн дараалалд шилжүүлнэ. Энэ хооронд CPU бусад процесст ашиглагдана.

Бүрэн зайлсхийж болох уу? Үгүй, гэхдээ маш багасгаж болно. Блоклогдох семафор, мутекс зэргийн `wait()`/`signal()` үйлдлүүд өөрсдөө дундын өгөгдлийг (семафорын утга, хүлээлтийн жагсаалт) өөрчилдөг тул атомар байх ёстой. Үүнийг хамгийн доод түвшинд богино spinlock эсвэл тасалдал хаах замаар хамгаалдаг. Ингэснээр завгүй хүлээлт программын урт эгзэгтэй мужаас хэдхэн зааврын урттай богино хэсэг рүү шилжинэ. Олон процессортой системд богино хугацааны завгүй хүлээлт нь контекст сэлгэлтээс хурдан тул зориуд хэрэглэдэг.

Дасгал 6.3

Асуулт. Spinlock нь яагаад нэг процессортой системд тохиромжгүй, харин олон процессортой системд түгээмэл хэрэглэгддэгийг тайлбарла.

Хариулт.

- **Нэг процессортой систем.** Түгжээг эзэмшиж буй урсгал болон түгжээ хүлээж эргэлдэж буй урсгал нэг CPU-г хуваалцана. Эргэлдэж буй урсгал CPU-г эзэлж байх хооронд эзэмшигч урсгал ажиллаж чадахгүй тул түгжээг чөлөөлж чадахгүй. Түгжээ зөвхөн квант дуусаж контекст сэлгэлт гарсны дараа л чөлөөлөгдөнө. Иймд эргэлдэх хугацаа бүхэлдээ дэмий үрэгдэнэ. Блоклогдох (унтах) түгжээ хэрэглэх нь дээр.
- **Олон процессортой систем.** Түгжээ эзэмшигч урсгал өөр цөм дээр зэрэг ажиллаж байгаа тул түгжээг удахгүй чөлөөлнө. Түгжээг богино хугацаанд эзэмшдэг бол хэдэн цикл эргэлдэх нь унтаж сэрэхэд шаардагдах хоёр контекст сэлгэлтээс хамаагүй хурдан. Ерөнхий дүрэм: түгжээ эзэмших хугацаа хоёр контекст сэлгэлтээс богино бол spinlock тохиромжтой. Иймд Linux, Windows-ийн кернел олон процессортой системд богино эгзэгтэй мужид spinlock өргөн ашигладаг.

Дасгал 6.4

Асуулт. Семафорын `wait()` ба `signal()` үйлдлүүд атомар байдлаар гүйцэтгэгдэхгүй бол mutual exclusion зөрчигдөж болохыг харуул.

Хариулт.

Семафорын тодорхойлолт:

```
wait(S)   { while (S <= 0) ; /* busy wait */  S--; }
signal(S) { S++; }
```

$S = 1$ (binary семафор, эгзэгтэй мужийг хамгаалж буй) үед P_0, P_1 хоёр процесс зэрэг `wait(S)` дуудаж, атомар биш гүйцэтгэгдсэн гэж үзье:

Алхам	Үйлдэл	S	Эгзэгтэй мужид
1	P_0: `S <= 0`? $\rightarrow$ худал, давталтаас гарна	1	—
2	контекст сэлгэлт	1	—
3	P_1: `S <= 0`? $\rightarrow$ худал, давталтаас гарна	1	—
4	P_1: `S--`	0	P_1
5	контекст сэлгэлт	0	P_1
6	P_0: `S--`	-1	P_0, P_1 **хоёулаа!**

Шалгалт ба хасалтын хооронд сэлгэлт гарсан тул хоёр процесс хоёулаа $S = 1$ гэж харж эгзэгтэй мужид орсон. Ингэснээр **mutual exclusion зөрчигдлөө**.

Мөн `S--`, `S++` өөрсдөө машины түвшинд унших $\rightarrow$ өөрчлөх $\rightarrow$ бичих гурван алхам тул атомар биш. Жишээ нь $S = 0$ үед хоёр процесс зэрэг `signal(S)` хийвэл хоёулаа 0-ийг уншиж, 1 гэж бичнэ. Нэг `signal` алдагдаж, хүлээж буй процесс хэзээ ч сэрэхгүй болно. Эсрэгээрээ `wait`-ийн хасалт алдагдвал утга хэтэрч, дахин олон процесс эгзэгтэй мужид орно. Иймд `wait()`, `signal()` заавал атомар байх ёстой.

Дасгал 6.5

Асуулт. n процессын хооронд mutual exclusion хэрэгжүүлэхэд binary семафорыг хэрхэн ашиглахыг харуул.

Хариулт.

n процесс `mutex` нэртэй, **эхний утга нь 1** байх binary семафорыг хуваалцана. Процесс P_i ($i = 1, \ldots, n$) бүр:

```
semaphore mutex = 1;          /* бүх n процессын дундын */

while (true) {
    wait(mutex);              /* орох хэсэг (entry section)  */
        /* эгзэгтэй муж */
    signal(mutex);            /* гарах хэсэг (exit section)  */
        /* үлдсэн хэсэг (remainder section) */
}
```

Яагаад ажиллах вэ:

- Эхний `wait(mutex)` дуудсан процесс утгыг $1 \to 0$ болгож эгзэгтэй мужид орно.
- Бусад $n-1$ процесс `wait` дуудахад утга 0 тул блоклогдож семафорын хүлээлтийн жагсаалтад орно.
- Эгзэгтэй мужаас гарсан процесс `signal(mutex)` хийнэ. Хүлээж буй процесс байвал нэгийг нь сэрээнэ, байхгүй бол утгыг 1 болгоно.
- Иймд аль ч мөчид эгзэгтэй мужид **зөвхөн нэг** процесс байна (mutual exclusion). Хүлээлтийн жагсаалт FIFO бол хүлээх хугацаа хязгаартай (bounded waiting) байж, starvation гарахгүй.

Дасгал 6.6

Асуулт. Уралдааны нөхцөл олон компьютерийн системд гарч болно. Банкны данс хөтөлдөг системийг авч үзье. Дансны үлдэгдэл `deposit(amount)` (хийх) ба `withdraw(amount)` (авах) гэсэн хоёр функцээр өөрчлөгдөнө. Нөхөр, эхнэр хоёр нэг данс хуваалцдаг гэж үзье. Нөхөр нь `withdraw()`, эхнэр нь `deposit()` функцийг зэрэг дуудна. Уралдааны нөхцөл хэрхэн үүсэх, түүнээс сэргийлэхэд юу хийж болохыг тайлбарла.

Хариулт.

Уралдааны нөхцөл. `balance = balance - amount` болон `balance = balance + amount` нь машины түвшинд унших $\to$ тооцох $\to$ бичих гурван алхамтай. Үлдэгдэл 1000, нөхөр 100 авч, эхнэр 200 хийнэ гэж үзье. Зөв үр дүн нь $1000 - 100 + 200 = \mathbf{1100}$:

Алхам	Нөхөр: `withdraw(100)`	Эхнэр: `deposit(200)`
1	`reg1 = balance` (1000)	
2		`reg2 = balance` (1000)
3	`reg1 = reg1 - 100` (900)	
4		`reg2 = reg2 + 200` (1200)
5	`balance = reg1` (900)	
6		`balance = reg2` (**1200**)

Эцсийн үлдэгдэл **1200** болж, авсан 100 алдагдсан. Хэрэв 5, 6-р алхмын дараалал солигдвол **900** болж, хийсэн 200 алдагдана. Үр дүн гүйцэтгэлийн дарааллаас хамаарч байгаа нь уралдааны нөхцөл юм.

Сэргийлэх арга. `balance`-д хандах кодыг эгзэгтэй муж болгож, нэг удаад зөвхөн нэг функц ажиллахыг баталгаажуулна:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void deposit(int amount)  { lock(&lock); balance += amount; unlock(&lock); }
void withdraw(int amount) {
    lock(&lock);
    if (balance >= amount) balance -= amount;   /* шалгалт ч эгзэгтэй мужид */
```

```
    unlock(&lock);
}
```

Мутексийн оронд binary семафор, монитор (Java-д `synchronized` метод), атомар заавар (`compare_and_swap`) эсвэл өгөгдлийн сангийн transaction (түгжээ) ашиглаж болно. `withdraw`-ийн ”үлдэгдэл хүрэлцэх эсэх” шалгалтыг ч түгжээн дотор хийх ёстой. Эс бөгөөс хоёр зэрэг авалт хоёулаа шалгалтыг давж, үлдэгдэл сөрөг болно. Энэ нь лабын 6.33-ын `decrease_count()`-тай яг ижил асуудал юм.